FTC 5206 The Knights of Ni Offseason Documentation

Han Lu

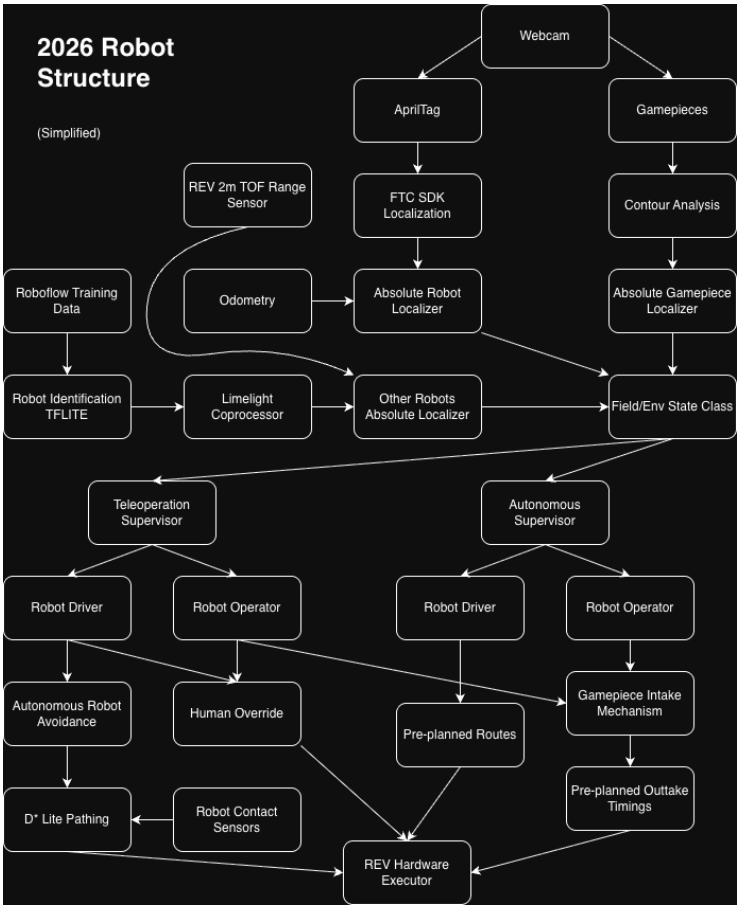
Pre-Release 0.2.4

Introduction

This text serves as the documentation for various hardware and software developments done in the offseason following Team 5206 The Knights of Ni's last tournament on Saturday, January 24 of the 2026 FTC DECODE Season. All code is linked to the appropriate GitHub and CAD to the appropriate Google Drive. Software's focus was improving localization and object recognition. Hardware's goal was to prototype common mechanisms that can be implemented next season and solving REV Control Hub disconnection issues. The following work was done and documented by Han Lu ('27), Team 5206's Hardware Lead.

Software

The primary goal of software during offseason was to implement fully autonomous driving, even during the traditionally driver-operated period. Several methods novel to our team had to be implemented, including the identification and localization of other robots through neural networks, planning paths around other robots in a dynamic environment, and multithreading. The table summarizes the structure of what we hoped to achieve this offseason.

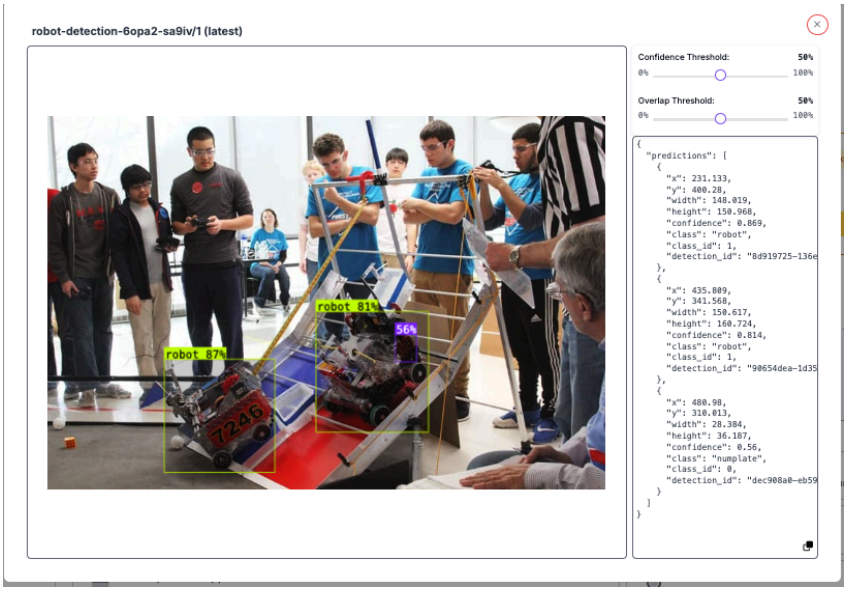


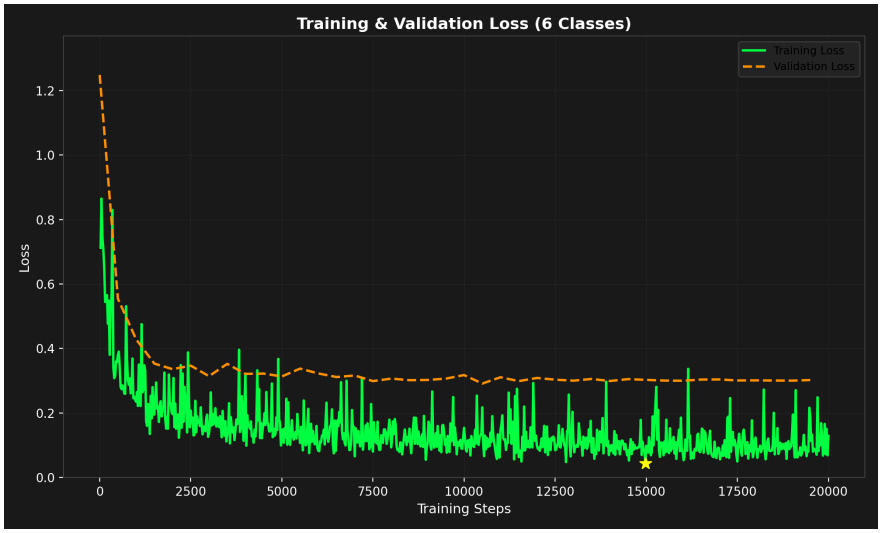
Robot Identification

It is common knowledge that the newly introduced Limelight Camera can be used for accurate turret alignment and localization. Combined with a traditional camera and odometry deadwheels, this data can be accurately computed to ascertain the absolute position of the robot on the field. Furthermore, cameras have always been used to locate game pieces for better intake and outake. However, it seems as if FTC teams have never attempted to locate other robots. We decided to tackle this problem because it would allow for autonomous drive during the driver operated period or augment the driver skills in avoiding other robots while navigating the field. The following documentation was written as the feature was first being developed.

According to 2026 Robot Construction Rule R704, the Limelight 3A is the only vision coprocessor that can be programmed. The feasibility of this project relies on offloading the computationally intensive task of robot identification via a neural network to this vision coprocessor. Then, the Control Hub may be used to interpret and perform regular functions.

The dataset used for training was taken from Roboflow Universe, courtesy of Mitchell Peck. This dataset contained 1.06k annotated images of robots from 2023 POWERPLAY, 2024 CENTERSTAGE, and 2025 INTO THE DEEP. These images were taken from online livestreams. The dataset was augmented with 15% brightness deviation, 2.5 px blur, and 0.1% noise to make the model more adaptive to field conditions. The total augmented set contained 2734 annotated images. For initial testing, a Roboflow nano model was used with a 2511 for training, 152 for validation, and 71 for testing, which took around 2 hours. With a confidence threshold of around 75% the testing model was able to pass all of the validation tests pulled from the internet.

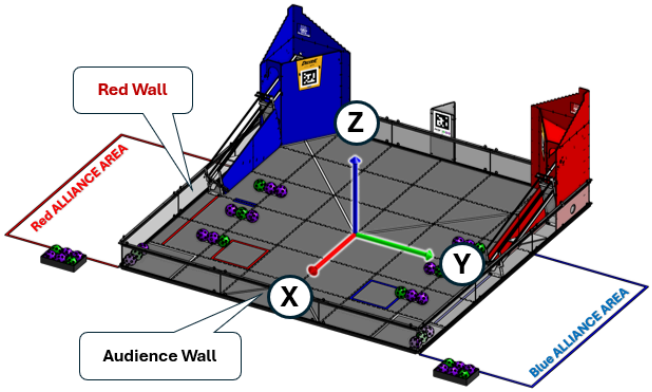




Using the Limelight Free Neural Network Trainer, a 20000 step TFLITE file, along with the labels.txt required, was generated for Coral/CPU computing, which was then deployed on our Limelight 3A. Since our model is a detector model, we used the following code to get its position on the screen:

```
List<DetectorResult> detections = result.getDetectorResults();
for (DetectorResult detection : detections) {
```

```
    String className = detection.getClassName(); // class label
    double x = detection.getTargetXDegrees(); // x coord on cam
    double y = detection.getTargetYDegrees(); // y coord on cam
    telemetry.addData(className, "at (" + x + ", " + y + ") degrees");
}
```



Now, we have the x and y coordinates, supposedly the center of the detected robot object, classified based on label.txt. To get the global pose of all the bots, it is useful to first define the coordinate system. For simplicity, let the relative position and absolute position use the same axes, defined by the FTC SDK. Now, let's find the relative position of the identified robot with respect to our robot. Our data does not have to be very precise, so let us first assume the robot is a point, then create an exclusion zone when actually implementing the drive code. Since the y-coordinate on the limelight is not really used for anything, assume we are talking about the field unless otherwise specified. This model was trained on 512 x 512 px preprocessing, using the 640 x 480 px for 90 FPS sounds reasonable. Let's first consider our robot at the origin, with the camera facing the positive x direction, meaning the absolute orientation is zero degrees. If the target robot is also on the positive x axis, then detection.getTargetXDegrees() = 0. If the target robot is at some (x_0, y_0), in the first quadrant, and has detection.getTargetXDegrees() = θ, the following must be true using trigonometry:

$$\frac{y_0}{x_0} = \tan\theta$$

The camera can never see things in the 2nd or 3rd quadrants and is constrained to -27.75º < θ < 27.75º because of its 54.5º horizontal field of view. Thus, when θ is negative, it is always the y value that is negative, not the x. This is important when solving for the actual position. Using a REV 2m time of flight (TOF) distance sensor, we also have the direct distance to the target robot:

$$\sqrt{x_0^2 + y_0^2} = d$$

Now, solving for the relative position is trivial:

```
List<DetectorResult> detections = result.getDetectorResults();
for (DetectorResult detection : detections) {
    String className = detection.getClassName();
    double d = getTOFdist(); // from 2m sensor
```

```
    double x = d * Math.cos(Math.toRadians(detection.getTargetXDegrees()));
    double y = d * Math.sin(Math.toRadians(detection.getTargetXDegrees()));
    telemetry.addData(className, "relative at (" + x + ", " + y + ")");
}
```

To find the absolute position of the target robot on the field, we first need to know the absolute position of *our* robot on the field. Conventional methods such as deadwheel odometry and AprilTag recognition have already implemented this feature in the SDK, so we will not cover it. Let's assume that the robot knows its absolute position, (x,y). Using the in-built IMU along with the aforementioned methods, we also know our yaw, μ. Again, we define the positive x axis to be $\mu = 0$ and $\mu > 0$ to be in the counter-clockwise direction towards the positive y-axis. If we first rotate the vector produced by the relative position by μ and add it to the absolute position of our robot, we can get the absolute position of the target robot:

```
List<DetectorResult> detections = result.getDetectorResults();
for (DetectorResult detection : detections) {
    String className = detection.getClassName();
    double d = sensorRange.getDistance(DistanceUnit.MM); // from 2m sensor
    double x = d * Math.cos(Math.toRadians(detection.getTargetXDegrees()));
    double y = d * Math.sin(Math.toRadians(detection.getTargetXDegrees()));

    Pos2D pos = robot.odo.getPosition();
    double xi = pos.getX(DistanceUnit.MM);
    double yi = pos.getY(DistanceUnit.MM);
    double yaw = imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.RADIANS);

    double vecx = x * Math.cos(yaw) - y * Math.sin(yaw);
    double vecy = x * Math.sin(yaw) + y * Math.cos(yaw);
    double absX = xi + vecx;
    double absY = yi + vecy;

    telemetry.addData(className, "absolute at (" + absX + ", " + absY + ")");
}
```

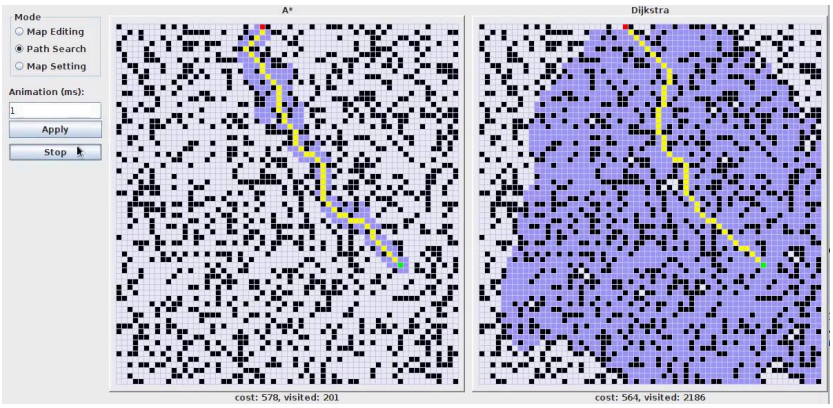
Now, we have successfully calculated the absolute position of all the bots on the field. The next step is to implement neural-network based pathing to avoid these target robots based on these coordinates or other strategic uses.

26 January, 2026.

Strategic Pathplanning

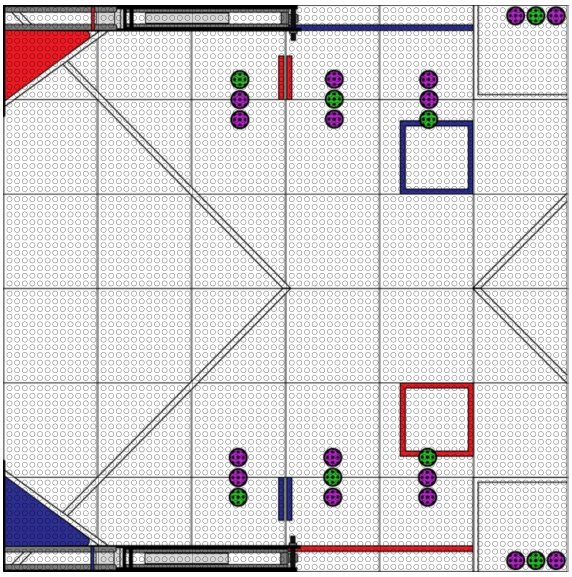
Our goal is to create a model that takes an array outputted by an interpreter to generate proper robot motion, whether that be through intended positions or motor power directly. Interpreter in this instance is just the program that combines all of the robot positions calculated above with gamepiece and current robot location to create a representation of the field state. For simplicity, we decided to begin with the simplest case, where the robot is a square moving in a 2D plane.

As an aside, we first tried to use reinforcement learning (RL) to implement pathing. The benefit to RL is its scalability and inherent generalizing nature. If the pipeline is successfully developed, it can be reused for any future FTC games and can be scaled to incorporate other objectives that the model must learn, such as picking up game pieces and scoring. Initially, we attempted to use the ml-agents framework from Unity, training the robot as if it was playing a video game. Although the path finding model was highly accurate after just a few epochs, the outdated nature of the framework disallowed us from exporting from the pytorch subsystem to a runnable onnx or tflite file. Next, we tried to use OpenAI's Gymnasium along with TensorFlow's tf-agents framework. We were not able to get a model fully working before scrapping the idea. The main concern with the RL approach was that, due to the limited computing power of the REV Control Hub, the model will be too slow. Thus, we shifted to non-inferential methods of pathing.



In this section, we are primarily concerned with obstacle avoidance, with the obstacle in question being other robots. Searching algorithms that find the best path have already been implemented, so we chose the one that best suits our needs. Since typically on the field there are not many obstacles (3 other robots at most), the best path is typically just a straight line. Thus, a greedy fastest first algorithm is the obvious choice. The classical choice for this type of algorithm is A* (pronounced A-star), but since we are interested in dynamic environments, we chose D* (D-star) Lite. The main difference between D* and A* is that the former modifies its trajectories based on newly available data. D* changes the path based on newly encountered obstacles, which would be useful since robots change positions while A*'s trajectory cannot be changed after computing. This is particularly useful since we want the robot to compensate for other robots playing defense, not just focusing on driving the most optimal path. Likewise, with D* we can define a limited range where the robot is able to obtain accurate information. Since we are using a 2m REV Distance Sensor, the range is limited to 2 meters, which, in optimal lighting conditions,

covers 47% to 94% of the field area. Because we have limited information, the algorithm must be able to adapt to new readings, thus highlighting another benefit of D*.



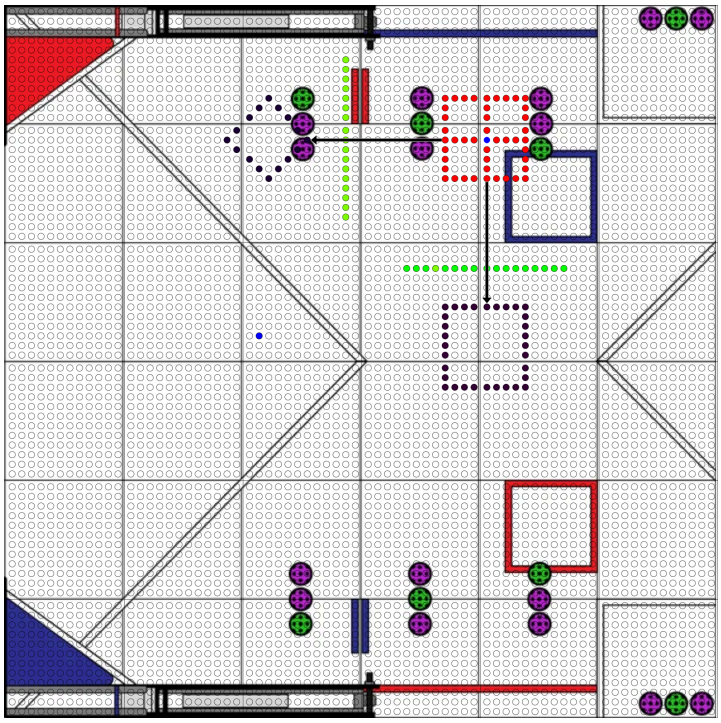
To implement this algorithm, we use Daniel Beard's DStarLiteJava repository. Pathfinding is node-based, so the field has to be discretized. Prelimilarily, consider a resolution of one inch per node, which leads to 20736 nodes on a 12ft x 12 ft field. Let the origin be node (0,0) and the positive x and y axes to be defined according to the FTC field, as shown before. Thus, anything on the field will have a coordinate ranging from (-72,-72) to (72,72). The basic usage of DStarLite Java is to first define the start and end nodes, update the impassable nodes, and execute pathfinding. Defining the start and end nodes is not very hard and is dependent on what units are outputted by the implementation of the previous section, so we will disregard that. The placement of other robots requires more careful treatment. To create impassable nodes, first set the boundaries for the game field. The pathfinder is allowed to be one node away from the properly impassable node, but physically this is not possible, since the robot is more than one square inch. In fact, it will occupy 18 by 18 nodes (if we disregard rotation). So, any exclusion zone has to factor this in by adding a 9 node extension. The extra inch may be used as error. Thus, in reality, the robot center that is traveling through the nodes can only travel from (-63, -63) to (63,63). We can then initially update the field with these exclusion zones. Note that the wall only needs to be one node thick.

```
DStarLite pf = new DStarLite();
pf.init(startX, startY, goalX, goalY);

for (int i = -64, i <= 64, i++){
   pf.updateCell(i, -64, -1); // -1 cost indicates it is impassable.
   pf.updateCell(i, 64, -1);
}
for (int i = -64, i <= 64, i++){
   pf.updateCell(-64, i, -1);
   pf.updateCell(64, i ,-1);
}

pf.replan(); // performing pathplanning.
List<State> path = pf.getPath();
for (State i : path){
```

```
    System.out.println("x: " + i.x, + " y: " + i.y);
}
```



Now, for simple pathfinding, we repeat this for all 3 other robots on the field. However, because of sensor limitations, more care must be taken. The range sensor can be rotated such that it always samples the "center" of the robot as viewed from the camera. However, this reading only gives one value. If we always sample the robot and the robot is close to a cubical shape, then from the robot's perspective it only needs to know that the robot is an impassable object. Thus, we can represent the robot as an 18 node wall centered on the "sample site". Again, we add 9 nodes to account for the size of our robot.

See figure. Nodes are displayed at half resolution. Our robot node is represented by the blue dots, with one being the starting and one being the goal. The red shows the actual size of the robot, noting well that the searching algorithm only believes that it is the size of the blue dot, requiring our use of bigger exclusion zones. The purple cubes are the opposing robots, while the green nodes are the impass nodes created by our algorithm. If the robots were always orthogonal to each other, identifying the exclusion nodes required would be trivial. However, in reality it is not that simple. For example, how does one create exclusion nodes for an opposing robot that is 45 degrees from our robot? Thus, we need a formula that takes our robot position (let this be denoted (x_1, y_1)) and one of the opposing robots (let this be denoted (x_2, y_2)) and outputs 36 nodes which we can input into the pf.updateCell function. Optimally, we would also not need to call the sensorRange.getDistance() function, since it might be called by another thread.

An astute observer might note that the green walls are unnervingly close. Indeed, if the opponent bots were a bit closer, the walls would connect and the program would believe that there is no way to pass. While the bot would probably be able to pathfind around this wall (e.g. going right if both of the opponents are on the left), it will be useful to have a general solution to this problem. Likewise, there should be a backup plan if the bot has to pathfind all around the field just to get to a point. This issue can be solved strategically or algorithmically. First, we have to consider *why* we want to get to that point. In this example, we need to get into the shooting zone to be able to score. Consequently, we are not really concerned with where we are in the shooting range, but just that we are in it. This solution will be easy to implement, but it is really dependent on

the specific application, which varies from year to year. To solve this problem algorithmically, we have to turn to a different search solution.

Specifically, a touch sensor array will be used. [Much more to come in the onseason...]

FTC Dashboard and Pedro Pathing

Contributors: Aanya Gupta ('27)

Since our previous endeavours led us to require proper field view, we decided to hit two birds with one stone and implement FTC Dashboard and Pedro Pathing for more effective localization and autonomous planning. The former was easy to implement, requiring only gradle script modifications. The second was tuned using dummy variables and mostly served to assist in software planning. No hardware modifications were completed at this time, as the main objective

Hardware

During our DECODE season, we have noticed that there were two major problems with our robot that was not season-specific, meaning we could improve them over the offseason. Most pressingly, we have found that our power distribution is very unstable and our control hubs often disconnect. Since our hardware is old, it was more sensitive to the touch-heavy game this year. Thus, we had to find ways to mitigate electro-static discharge (ESD) and increase power reliability. The second issue we faced was that our drive train was somehow slower than everyone else, even though we used the latest 104 mm diameter chassis kit from GoBilda. Software has already contributed to this by implementing PedroPathing, but we decided that testing out swerve for this offseason would be helpful as well.

Rewiring

Swerve

There are many benefits to swerve drive over traditional mecanum drive, including better traction, higher customizability, and higher top speed. On the other hand, there are notable disadvantages to swerve drive, the most notable being complexity and cost. Swerve was a very CAD (computer-aided design) heavy project because of its complexity and small scale. Indeed, the swerve we ended up with had a total footprint of less than 6 inches in each direction, which is significantly smaller than FRC swerve drives. Notable differences from the 2024 swerve drive design include a smaller form factor, less backlash, and the incorporation of Axon Robotics bevel gears, a technology unavailable when I first designed swerve.

The initial inspiration was from a TerraVoxel video, showcasing a swerve concept. The gearing is directly from this video, although it is pretty much the standard in the FTC swerve scene. The gearing is as follows. First,

a 6000 RPM D-shaft bare motor is geared 1:1 to the top of the pod by using two 3D printed 20 tooth GT2 pulleys with a 2mm, 76 tooth belt. This length was chosen to minimize size, placing the motor as close to the wheels as possible. The motion is transferred through a 5mm hex shaft to a 12 tooth gear, which is then connected to another shaft through a 30 tooth gear for a cumulative gear reduction of 1:2.5. The 30 tooth gear transfers the motion further to an axon pinion gear through a 8mm hex shaft. The axon pinion meshes with the axon bevel gear with a reduction of 1:2.88, for a cumulative reduction of 1:7.2. So, the wheel spins at around 833 rpm. The wheel itself is around 62 mm in diameter, allowing for a theoretical top speed of around 9 feet per second, which is significantly higher than mecanum's 5.35 in standard configuration (from the GoBilda kit, which we used for this season).

Acknowledgements

Team 5206 The Knights of Ni would like to acknowledge our sponsors Stanford Online High School and Apple for their continued financial support of our robotics endeavours. We would also like to acknowledge the graduating seniors (class of 2026) and thank them for their services and contributions to the Team, however long they have been active. Finally, we would like to acknowledge the families and mentors that have enabled us to attend Stanford OHS and work on the robot.

Appendix

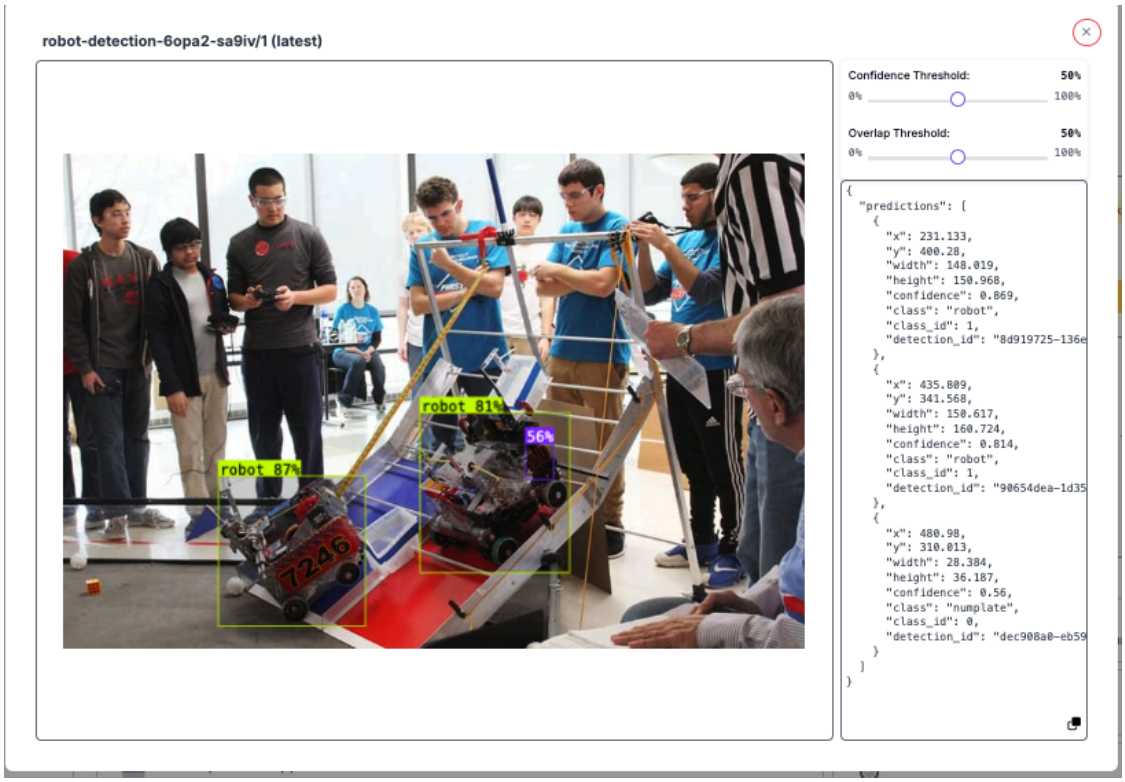
Extra information, such as figures and data that could supplement conversing with judges.

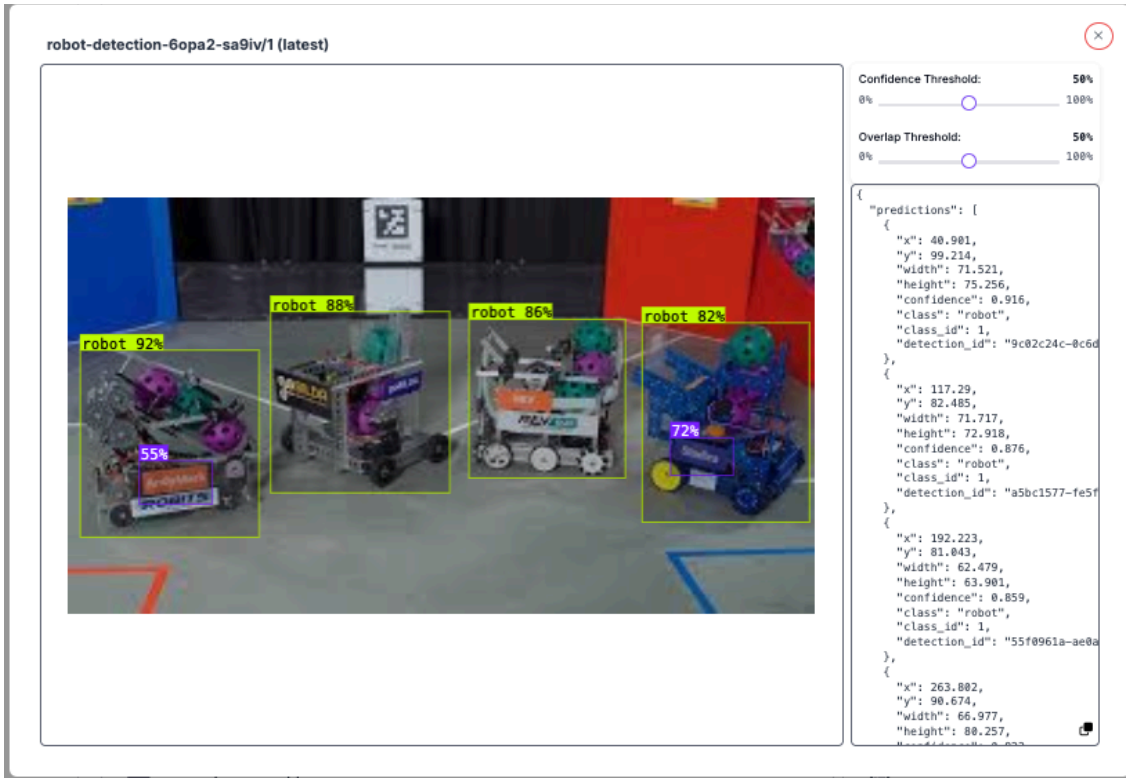
Figures

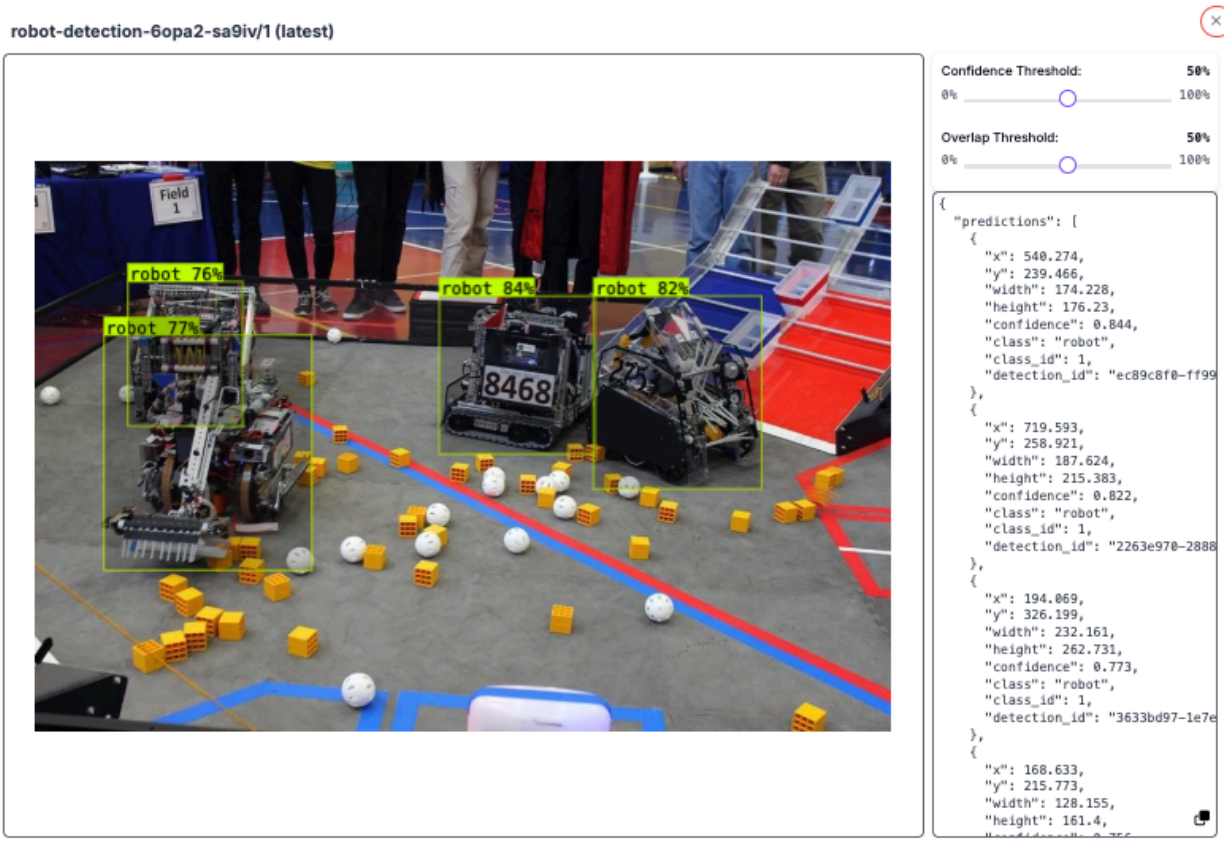
Collection of figures, included and not included in the main text.

Software - Robot Recognition

Validation

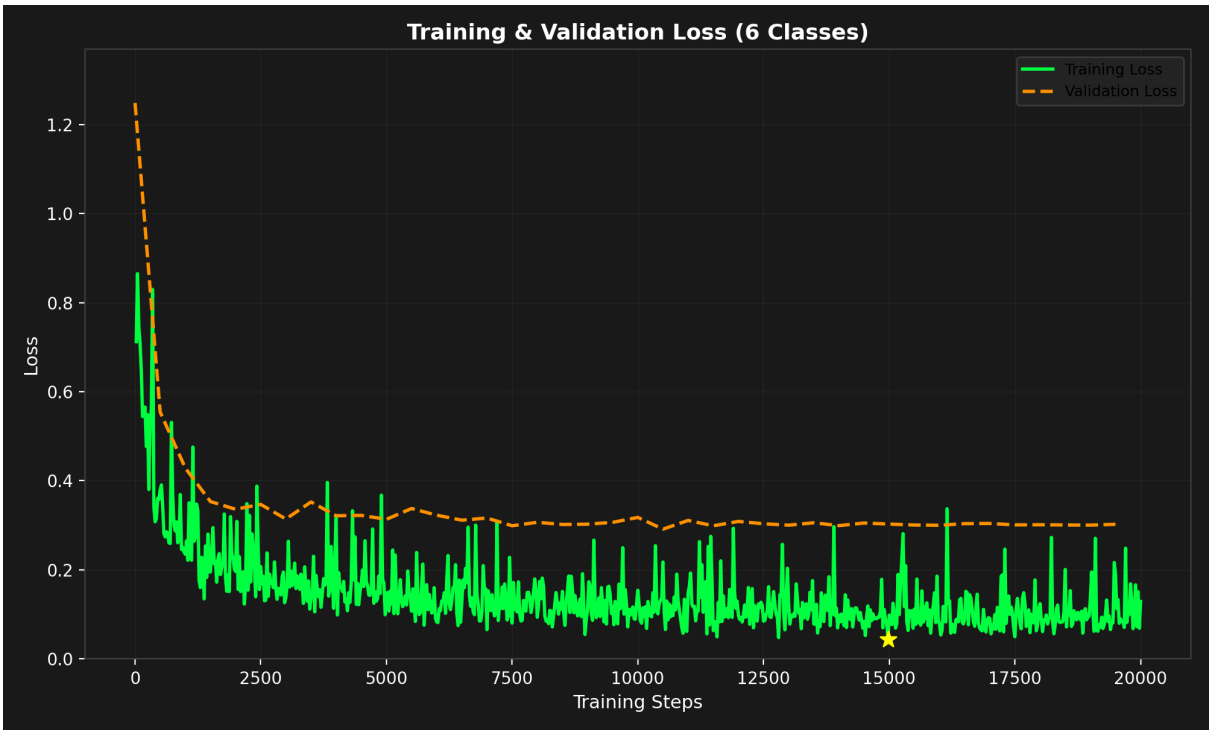



robot-detection-6opa2-sa9iv/1 (latest)
Confidence Threshold: 50%
0% 100%
Overlap Threshold: 50%
0% 100%
robot 92%
robot 88%
robot 86%
robot 82%
55%
72%
{
"predictions": [
{
"x": 40.901,
"y": 99.214,
"width": 71.521,
"height": 75.256,
"confidence": 0.916,
"class": "robot",
"class_id": 1,
"detection_id": "9c02c24c-0c6d
},
{
"x": 117.29,
"y": 82.485,
"width": 71.717,
"height": 72.918,
"confidence": 0.876,
"class": "robot",
"class_id": 1,
"detection_id": "a5bc1577-fe5f
},
{
"x": 192.223,
"y": 81.043,
"width": 62.479,
"height": 63.901,
"confidence": 0.859,
"class": "robot",
"class_id": 1,
"detection_id": "55f0961a-ae0a
},
{
"x": 263.802,
"y": 90.674,
"width": 66.977,
"height": 80.257,


robot-detection-6opa2-sa9iv/1 (latest)
Confidence Threshold: 50%
0% 100%
Overlap Threshold: 50%
0% 100%
Field 1
robot 76%
robot 77%
robot 84%
robot 82%
8468
{
"predictions": [
{
"x": 540.274,
"y": 239.466,
"width": 174.228,
"height": 176.23,
"confidence": 0.844,
"class": "robot",
"class_id": 1,
"detection_id": "ec89c8f0-ff99
},
{
"x": 719.593,
"y": 258.921,
"width": 187.624,
"height": 215.383,
"confidence": 0.822,
"class": "robot",
"class_id": 1,
"detection_id": "2263e970-2888
},
{
"x": 194.069,
"y": 326.199,
"width": 232.161,
"height": 262.731,
"confidence": 0.773,
"class": "robot",
"class_id": 1,
"detection_id": "3633bd97-1e7e
},
{
"x": 168.633,
"y": 215.773,
"width": 128.155,
"height": 161.4,

include pics from limelight too

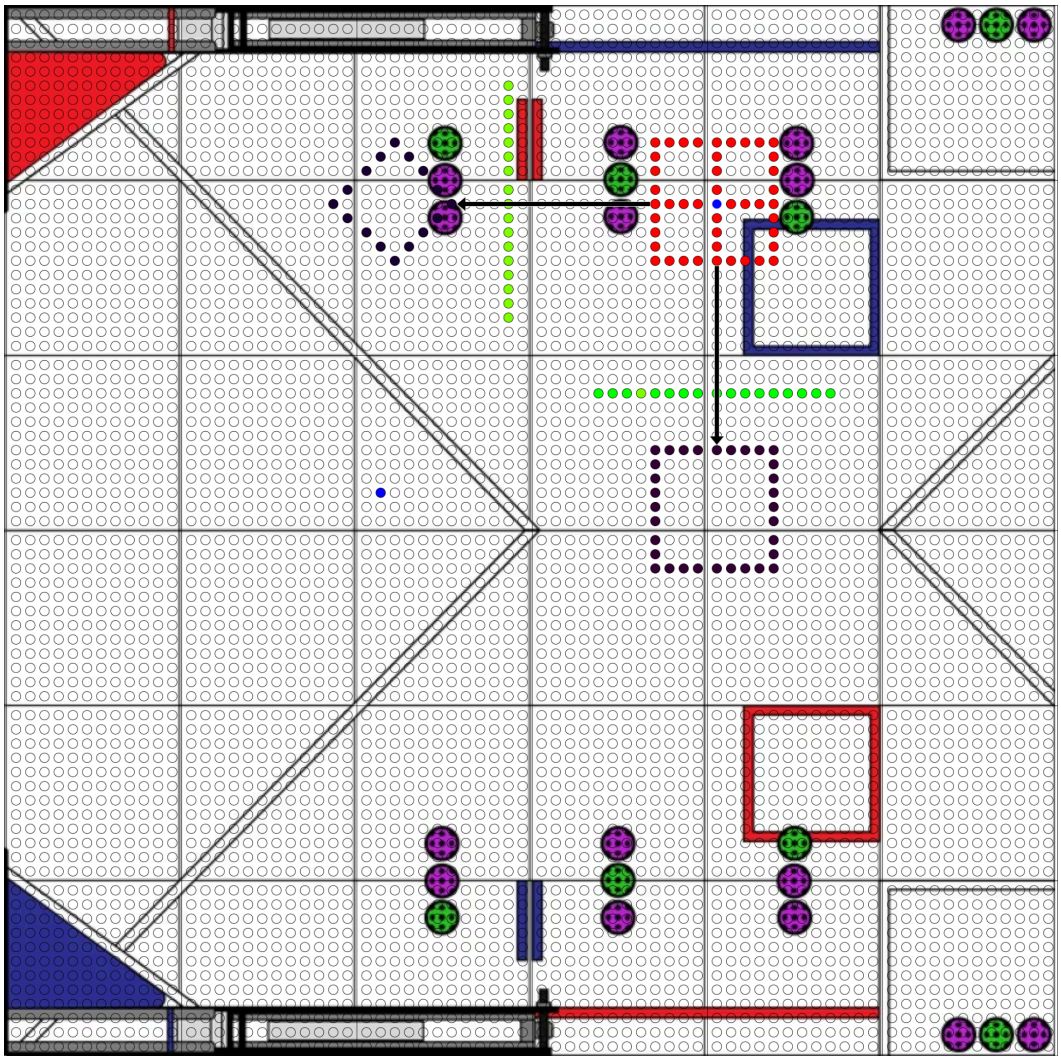
Training





Software - Pathplanning

Nodes



Works Cited

https://www.sciencedirect.com/science/article/pii/S1874490724001782
https://developer.nvidia.com/isaac/sim
https://github.com/newton-physics/newton
https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/index.html
https://docs.limelightvision.io/docs/docs-limelight/getting-started/limelight-3a
https://docs.limelightvision.io/docs/docs-limelight/pipeline-neural/training-your-own-detector
https://www.youtube.com/watch?v=SjVqOqEXXrY
https://docs.google.com/document/d/10sXEhzFRSnvFcl3XxNGhnD4N2SedqwdAvK3dsihxVUA/edit?tab=t.0#heading=h.2ye70wns7io3
https://docs.qualcomm.com/doc/80-88500-4/topic/145_TensorFlow.html
https://github.com/bulletphysics/bullet3
https://mrsiefensrobotemporium.com/blogs/2022/October/myFirstFTCROSbotpt1.html
https://www.youtube.com/watch?v=H2uUvhSKpWI
https://github.com/tensorflow/agents
https://www.youtube.com/watch?v=52DTXidSVWc&list=PLQY2H8rRoyvxWE6bWx8XiMvyZFgg_25Q_
https://github.com/vwxyzjn/cleanrl
https://ieeexplore.ieee.org/document/1435479/
https://ieeexplore.ieee.org/document/10853212
https://www.redblobgames.com/pathfinding/a-star/implementation.html
https://gamedev.stackexchange.com/questions/197165/java-simple-2d-grid-pathfinding
https://github.com/Sollimann/Dstar-lite-pathplanner
https://github.com/daniel-beard/DStarLiteJava
https://github.com/LynnOwens/starlite
https://www.youtube.com/watch?v=g024lzsknDo
https://drive.google.com/file/d/1EBF2eZM41xz5Ja1FQ0Joo93slC0XDOQP/view?usp=sharing
https://www.youtube.com/watch?v=qTGq6p0JgFw
https://doi.org/10.48550/arXiv.2409.16950